# Python Engineering - Week 01 / Class 04

## EX11 - Taylor e^x as a Reusable Function

From a working loop to a reusable mathematical tool

**Current path:** 01_python_bootcamp / week_01_logic / class_04_functions / exercises / ex11_taylor_exp_function.py

## 1. Exercise goal

In EX07 we built the Maclaurin approximation for e^x with a loop. In EX11 we are not relearning Taylor: we are **encapsulating that algorithm inside a function** so it can be reused many times without copying code.

**e^x ~= sum_(n=0)^N x^n / n!**

We want to move from a linear script to a simple interface:

```
approximation = taylor_exp(x, order)
```

Think of the function as a mathematical machine:

**(x, order) -> taylor_exp(...) -> e_N(x)**

## 2. What happens inside the function

Each call creates its own **local scope**. Algorithm variables such as `term`, `factorial`, and `approximation` belong to that call and do not need to exist as globals.

```
def taylor_exp(x, order):
    approximation = 0.0
    factorial = 1

    for n in range(0, order + 1):
        term = x**n / factorial
        approximation += term
        factorial *= n + 1

    return approximation
```

The key idea is that **factorial carries state between iterations**. At the beginning of iteration n, it should contain n!. At the end, we prepare it for the next iteration.

| n | factorial on entry | term | factorial prepared |
|---|---|---|---|
| 0 | 1 = 0! | 1 | 1 = 1! |
| 1 | 1 = 1! | x | 2 = 2! |
| 2 | 2 = 2! | x^2 / 2 | 6 = 3! |
| 3 | 6 = 3! | x^3 / 6 | 24 = 4! |

## 3. Parameters, arguments, and reuse

The definition uses **parameters**:

```
def taylor_exp(x, order):
```

and each call supplies **arguments**:

```
a = taylor_exp(0.5, 5)
b = taylor_exp(0.5, 10)
c = taylor_exp(2.0, 10)
```

All three calls run the same algorithm, but each call has its own local variables. This is the practical benefit of encapsulation.

## 4. Validation inside the function

Here `order` means the highest polynomial order included, so it must be a non-negative integer:

**order in Z, order >= 0**

Keep the EX09/EX10 pattern: invalid input may terminate the function with `return None`; the caller decides what to do next.

No special branch is needed for order = 0: the general loop already generates x^0 / 0! = 1.

## 5. Docstrings: document the function where it lives

From this class onward, we start documenting individual functions with a **docstring** placed directly under `def`:

```
def taylor_exp(x, order):
    """
    Approximate e^x using the Maclaurin series.

    Parameters
    ----------
    x : float
        Point where the exponential is evaluated.
    order : int
        Highest polynomial order included.

    Returns
    -------
    float or None
        Taylor approximation, or None for invalid order.
    """
```

A docstring does not affect the calculation; it makes the function explain what it receives, does, and returns.

## 6. Basic type hints (optional in EX11)

We can annotate expected types without forcing Python to enforce them. For Python 3.9, a compatible form is:

```
from typing import Optional

def taylor_exp(x: float, order: int) -> Optional[float]:
    ...
```

**Important:** type hints are documentation and tooling support; Python does not automatically coerce invalid values or reject calls just because of the hint.

## 7. What stays inside and outside

**Inside:** order validation, Taylor loop, accumulators, and return.

**Outside:** concrete test values, comparison with math.exp(), error calculations, and prints.

```
x_value = 0.5
order = 5

approx = taylor_exp(x_value, order)

if approx is not None:
    exact = exp(x_value)
    absolute_error = abs(exact - approx)
    print(...)
```

This separates the **mathematical engine** from the **experiment presentation and validation**.

## 8. Required test cases

| Case | x | order | Check |
|---|---|---|---|
| A | 0.5 | 0 | result = 1 |
| B | 0.5 | 5 | reasonable approximation |
| C | 0.5 | 10 | smaller error than order=5 |
| D | 2.0 | 10 | reuse with another x |
| E | 1.0 | -1 | return None |
| F | 1.0 | 3.5 | return None |

## 9. What NOT to use yet

Do not use scipy, numpy.exp, or another ready-made function to generate the approximation. `math.exp()` is only an independent reference for validation.

SymPy is not needed in this version either: we now want a **numeric reusable function** that receives a concrete x and returns a number.

## 10. Learning Tracker - EX11

| Concept | Goal |
|---|---|
| Function with internal loop | Consolidate encapsulation |
| Local scope | Avoid hidden global dependencies |
| return | Deliver value to the caller |
| Docstring | Document the interface |
| Type hints | Optional introduction |

| | |
|---|---|
| Reuse | Change x and order without copying code |
| Independent validation | Compare with math.exp() |

**Mission:** create `ex11_taylor_exp_function.py`. I want a self-contained function with no dependence on global x, N, factorial, or accumulators, plus several calls from the main program.